Clautolisp Implementation Roadmap

Codex

Purpose

This document is the architectural plan for bringing `clautolisp` from its current Phase-6 state (reader + evaluator + small core builtins, dialect-aware) to broad coverage of the AutoLISP / Visual LISP language family documented in `autolisp-spec`.

It distinguishes:

- **function coverage** — adding pure-language builtins that do not touch a CAD host (numeric, list, string, control, error);
- **host coverage** — adding the host-facing surface that real AutoLISP code depends on (entities, selection sets, tables, sysvars, command dispatch, graphics, dialogs, reactors);
- **backends** — the runtime swappable implementations of the host surface: a deterministic **MockHost**, an optional **GuiHost**, and a future **LiveHost** adapter to BricsCAD or AutoCAD.

The plan is intentionally headless-first: every layer must be testable without a graphical CAD process. A GUI backend is a later opt-in, not a prerequisite.

The plan is intentionally driven by the spec, not by an internal wishlist: each phase below references the chapters of the `autolisp-spec` document that define its scope.

Strategic Position

The architectural choice from chapter 1 of the spec stands:

1. **Implement AutoLISP as a Common-Lisp evaluator, not as a translation into Common Lisp.**
2. **Common Lisp is the implementation and extension language.** It does not provide the semantics.
3. *Host effects go through a single abstract interface (the HAL — Host Abstraction Layer) so that backends can be swapped at session-creation time.*

The dialect descriptor introduced in Phase 6 (`strict`, `autocad-2026`, `bricscad-v26`) controls **language-level** product divergence. The HAL choice is **orthogonal**: a dialect describes what the source code is allowed to mean; a backend describes where its host effects land.

Layered Architecture

```
+----------------------------------------+
|  AutoLISP source (your application)    |
+----------------------------------------+
|         Reader  (autolisp-reader)      |   chapter 2
+----------------------------------------+
|        Evaluator (autolisp-runtime)    |   chapter 3
+----------------------------------------+
|     Builtin registry (-builtins-core) |   chapters 4-7, 9-15
+----------------------------------------+
|  HAL - Host Abstraction Layer (NEW)    |   chapters 16, 18-22
+----------------------------------------+
|   Backend: MockHost | GuiHost (opt)    |
|            LiveHost (deferred)         |
+----------------------------------------+
```

The reader, runtime, and core builtins exist today and are sufficient for any program whose surface is values + control flow. The HAL is the next architectural addition; everything beyond it is backend-specific work.

Data Model — Inferred From the CAD Specifications

Most AutoLISP host functions operate on a small set of recurring data shapes. The runtime needs explicit Common-Lisp structures for each, even when the active backend stores nothing real behind them.

Entity (ENAME)

A handle naming a single CAD object. AutoLISP code obtains an ENAME from `entlast`, `entnext`, `ssname`, etc., and treats it opaquely. It is round-tripped through `entget` (returns a **DXF group-code list** — a list of dotted pairs of group-code integers and values) and `entmod` / `entupd` / `entdel` / `entmake` / `entmakex`.

```
(defstruct entity-handle
  (id      (gensym "ENT-") :type t)   ; opaque key into the backend
  (kind    nil :type symbol)          ; :line / :circle / :insert / ...
  (block   nil)                       ; owning block-table-record handle
```

```
  (layer   nil)                        ; layer-table-record handle
  (data    nil :type list)             ; cached DXF group-code list
  (deleted nil :type boolean))
```

The DXF group-code list (`data`) is the primary AutoLISP-visible representation: `(0 . "LINE") (5 . "1A") (8 . "0") (10 5.0 5.0 0.0) (11 7.0 5.0 0.0)` etc. Every entity-modifying builtin works against that shape; the backend is free to keep additional fields (graphic state, ObjectId, etc.) on the side.

Selection set (PICKSET)

A bag of ENAMEs with insertion-order-preserving identity. Built by `ssget` from filter lists; manipulated by `ssadd`, `ssdel`, `sslength`, `ssname`. `ssgetfirst` / `sssetfirst` are pickfirst hooks.

```
(defstruct pickset
  (id      (gensym "SS-") :type t)
  (members nil :type list))           ; ordered list of entity-handle
```

Tables (block, layer, ltype, style, view, ucs, vport, dimstyle, appid)

Symbol tables in DXF terminology. `tblnext`, `tblsearch`, `tblobjname`. `entget` on a table-record returns the same DXF list shape.

```
(defstruct symbol-table-record
  (id      (gensym "TBL-") :type t)
  (kind    nil :type symbol)          ; :block / :layer / :ltype / ...
  (name    "" :type string)
  (data    nil :type list))
```

The session keeps one **symbol-table** per kind, indexed by name.

Dictionaries

Generic key→object containers anchored at the named-object dictionary. `namedobjdict`, `dictsearch`, `dictnext`, `dictadd`, `dictremove`, `dictrename`. Dictionary entries can be entities, xrecords, or sub-dictionaries.

```
(defstruct dictionary
  (id      (gensym "DICT-") :type t)
  (entries (make-hash-table :test #'equal)))   ; string -> handle
```

Symbol tables for system variables

`getvar` / `setvar` access named state. `initget` / `getstring` / `getpoint` etc. read interactive input. The session needs a sysvar table:

```
(defstruct sysvar-table
  (entries (make-hash-table :test #'equalp)))   ; name -> sysvar-cell

(defstruct sysvar-cell
  (name    "" :type string)
  (kind    nil :type symbol)          ; :integer / :real / :point / :string / :short
  (value   nil)
  (read-only-p nil :type boolean))
```

SAFEARRAY / VARIANT

The Visual LISP COM bridge. SAFEARRAYs are typed multi-dimensional arrays; VARIANTs tag a value with its COM type. Modeled now (the runtime already has `autolisp-safearray`, `autolisp-variant`, `autolisp-vla-object` stubs); operators in the `vlax-*` family will fill them in.

VLA-object

A handle to a COM-exposed object (Layer, Block, Document, etc.). The runtime stores an opaque ID into a per-session table; the backend decides what the ID means.

```
(defstruct vla-object-handle
  (id          (gensym "VLA-") :type t)
  (kind        nil :type symbol)      ; :document / :layer / :block / :line / ...
  (entity-back nil)                   ; ename if applicable
  (released-p  nil :type boolean))
```

Layer states

Persistent named snapshots of layer-property bundles. `layerstate-*` family.

Reactors

Object-, document-, and editor-level callback registrations. Modeled per `vlr-*` entry: a callback list keyed by event class.

DCL dialog

The Dialog Control Language exposes a tile tree built from a .dcl file. Out of scope for the headless-first phase; tracked here so the data shape is visible (tile, action, list, image controls).

The Host Abstraction Layer (HAL)

Goal

Decouple the **what** (what the AutoLISP builtin does) from the **where** (which CAD backend stores or executes its effect).

Shape

A small set of generic functions per data domain. Each backend provides a method on every operation it supports; unsupported operations signal a uniform `:host-not-supported` runtime error.

```
(defgeneric host-entget        (host ename))
(defgeneric host-entmod        (host ename group-code-list))
(defgeneric host-entmake       (host group-code-list))
(defgeneric host-entdel        (host ename))
(defgeneric host-entlast       (host))
(defgeneric host-entnext       (host ename))

(defgeneric host-ssget         (host filter &key mode))
(defgeneric host-ssadd         (host pickset ename))
(defgeneric host-ssname        (host pickset index))
(defgeneric host-sslength      (host pickset))

(defgeneric host-tblsearch     (host table-kind name))
(defgeneric host-tblnext       (host table-kind &key rewind))

(defgeneric host-getvar        (host name))
(defgeneric host-setvar        (host name value))

(defgeneric host-command       (host argv))   ; the (command ...) builtin
(defgeneric host-prompt        (host string))
(defgeneric host-getstring     (host prompt &key controls))
(defgeneric host-getpoint      (host prompt &key base controls))
(defgeneric host-getreal       (host prompt &key controls))
(defgeneric host-getint        (host prompt &key controls))

(defgeneric host-grdraw        (host from to colour highlight))
(defgeneric host-grtext        (host kind ...))
(defgeneric host-grclear       (host))
```

Per-session backend

The `runtime-session` struct (autolisp-runtime/source/model.lisp) gains a slot:

```
(defstruct runtime-session
```

```
...
(host nil))
```

A new constructor argument `:host` defaults to a **NullHost** singleton that errors on every operation; production callers pass a MockHost or, eventually, a LiveHost.

Error codes

Existing runtime error codes (`:invalid-list-argument`, etc.) extend with:

- `:host-not-supported` — backend does not implement this operation
- `:host-not-connected` — operation requires a live CAD process
- `:host-protocol-error` — backend returned a malformed reply

Backends

NullHost

Trivial. Every generic function method signals `:host-not-supported`. Default for the strict and other test sessions; preserves current behaviour.

MockHost

In-memory deterministic CAD-database substitute. Owns:

- one `entity-table` (hash-table from ename id → entity-handle)
- one `pickset-table`
- one `vla-object-table`
- per-kind ~symbol-table~s for blocks, layers, ltypes, styles, views, ucs, vports, dimstyles, appids
- one `named-object-dictionary`
- one `sysvar-table` pre-populated with conservative defaults
- one `prompt-stream` (input source for `getstring` etc.)
- one `display-log` (records `grdraw` / `grtext` calls for inspection from tests)

The MockHost is the production runtime substrate for every clautolisp scenario that does not have a live CAD process behind it. It must be deterministic: the same input script under the same dialect, the same MockHost initial state, and the same `prompt-stream` must produce the same output and the same final database state.

GuiHost (optional, future)

A standalone Qt or web view that talks to clautolisp through the HAL via a JSON / S-expression protocol. Headless-test-friendly: the GUI is an observer of MockHost state, not a substitute for it.

LiveHost

A bridge to BricsCAD or AutoCAD. Speculative for now. Two plausible shapes:

- **Stdio bridge** — clautolisp drives an interactive AutoLISP REPL inside a real CAD process and reads back tagged S-expressions. Closest to the existing `autolisp-script` harness used for spec probes.
- **COM bridge** — Windows-only; the runtime drives the CAD process through its OLE Automation interface. More direct but Windows-bound.

Either way, LiveHost reuses the HAL contract: existing AutoLISP source runs unchanged when the backend is swapped.

Phased Roadmap

Numbering continues from Phase 6 (the Lisp-1, dynamic-scope, and general-semantics work that closed in 2026-04-26).

Phase 7 — Function-Coverage Round-Out

Goal: implement every host-independent builtin so that pure-data AutoLISP source loads cleanly. No HAL changes; the existing NullHost remains the default.

Scope (drives by `autolisp-spec` chapters 4-7, 9-13):

- Numeric / math: =sqrt, exp, log, log10, sin, cos, tan, asin, acos, atan, atan2 form, expt, mod, abs (already), random, floor, ceiling, round, fix (already)=
- Bitwise: `boole, ~ (already), logand (already), logior (already), logxor, lsh (already)`
- List: =append (already), reverse (already), nth (already), member (already), assoc (already), last (already), length (already), list (already), cons (already), subst (already), vl-list-length, vl-position, vl-remove (already), vl-sort, vl-sort-i, distance, remove= and the smaller corners.
- String: family already substantially in place; round out `vl-string-elt`, `vl-string-translate`, `vl-string-position` (done), `snvalid`, `wcmatch`, `xstrcase`.
- Predicate: `atom` (done), `numberp` (done), `zerop` (done), `minusp` (done), `listp` (done), `vl-consp` (done), `vl-symbolp` (done), `atoms-family`.
- Symbol: `boundp` (done), `vl-symbol-name` (done), `vl-symbol-value` (done), `set` (done), `atoms-family` (todo).
- I/O: `princ`, `prin1`, `print`, `terpri`, `read-line`, `read-char`, `write-line`, `write-char`, `open`, `close` (all done).
- Error / control: `vl-catch-all-apply` (done), `exit` / `quit` (done), `*error*` hook (done), `setfunhelp`, `vl-bt`, `vl-bt-on`, `vl-bt-off`.

Deliverables:

- one or two new sections in `autolisp-builtins-core/source/api.lisp`

- one new test file per family in `autolisp-builtins-core/tests/`
- coverage table (autogenerated from `core-builtins` vs the spec entries) in this document

Phase 8 — HAL Foundation

Goal: introduce the abstract host interface and the NullHost default. No behaviour change; a structural step.

Scope:

- Add `host` slot to `runtime-session` (defaulting to `*null-host*`).
- Define the generic-function surface in a new module `autolisp-host` depending on `autolisp-runtime`.
- Move the existing host-error code keywords into the new module.
- Re-route `find-core-builtin` entries that call out to the host through the HAL methods (rather than inline `error` calls).
- Add a `--host` CLI flag selecting MockHost / NullHost (default NullHost for backwards compatibility).

Phase 9 — MockHost Data Structures

Goal: the in-memory substrate for the entity / pickset / table / sysvar layers. No host operations yet; only the data carriers.

Scope:

- New module `clautolisp/autolisp-mock-host/` depending on `autolisp-runtime` and `autolisp-host`.
- Structs as sketched in section "Data Model".
- One pre-populated `sysvar-table` with the conservative subset (CMDECHO, CECOLOR, CLAYER, ANGBASE, ANGDIR, AUNITS, OSMODE, USERR1..R5, USERI1..I5, USERS1..S5).
- Snapshot / restore helpers for tests.
- Test fixtures: empty MockHost, single-line MockHost, layer-set MockHost.

Phase 10 — MockHost Entity API

Goal: implement the entity-level HAL methods against MockHost.

Scope (drives by `autolisp-spec` chapter 16):

- `entget` / `entmod` / `entdel` / `entupd`
- `entmake` / `entmakex` (with type-validating dispatch on the group-0 marker)
- `entlast` / `entnext`
- `handent`
- `snvalid` at the host level (checking against current table-record names)

Pre-populate MockHost with the standard tables ($\text{BLOCK}_{\text{RECORD}}$ with `"*Model_Space"` and `"*Paper_Space"`, LAYER with `"0"`, LTYPE with `"BYBLOCK" / "BYLAYER" / "Continuous"`, etc.).

Phase 11 — MockHost Selection / Tables / Sysvars

Goal: selection sets and table walkers.

Scope:

- `ssget` (with the AutoCAD filter-list grammar — `"X"` for boxed, `"_X"` for one-time, the dotted-pair filter list, the `ssget "X" '((0 . "LINE"))` shape)
- `ssadd`, `ssdel`, `ssname`, `sslength`, `ssmemb`
- `ssgetfirst` / `sssetfirst`
- `tblsearch`, `tblnext`, `tblobjname`
- `getvar`, `setvar` — with correct type coercion per sysvar kind

Phase 12 — Headless Interaction Channel

Goal: let real AutoLISP scripts that read user input run in batch.

Scope:

- `getstring`, `getreal`, `getint`, `getpoint`, `getcorner`, `getdist`, `getangle`, `getorient`, `getkword` — all read from a user-supplied stream attached to MockHost.
- `initget` — input-control flags applied to subsequent `get` calls.
- `prompt`, `princ` — write to MockHost's prompt stream.
- `grread` — minimal headless model (immediate eof / configurable scripted events).

The CLI gains a flag `--mock-input PATH` pointing at a script-driven response file, so a test rig can feed deterministic answers.

Phase 13 — VLA / VLAX Foundations

Goal: the COM compatibility surface, headless-first.

Scope (drives by `autolisp-spec` chapter 19):

- `vlax-create-object`, `vlax-get-object`, `vlax-release-object` (against a MockHost-managed COM-object table).
- `vlax-get-property`, `vlax-put-property`, `vlax-invoke-method` with documented method-name mapping.
- `vlax-make-safearray`, `vlax-safearray-fill`, `vlax-safearray-get-element`, `vlax-safearray-put-element`, `vlax-safearray->list`.
- `vlax-make-variant`, `vlax-variant-type`, `vlax-variant-value`, `vlax-variant-change-type`.

- `vlax-import-type-library` — implementation-defined; declarative description of mock-COM type libraries.
- `vla-*` wrappers — generated from a small declarative table of COM dispatch.

Phase 14 — Reactors

Goal: the Visual LISP reactor surface.

Scope (`autolisp-spec` chapter 22):

- All `vlr-*` functions: registration, callback dispatch, owner bookkeeping, persistence on the named-object dictionary.
- A MockHost-side event injection helper for tests (`mock-host-fire-event :command-will-start ...`).

Phase 15 — DCL and Optional GUI

Goal: dialog support, optional GUI backend.

Scope:

- DCL parser + tile model (`load_dialog`, `new_dialog`, `start_dialog`, `done_dialog`, `unload_dialog`).
- `action_tile`, `set_tile`, `get_tile`, `mode_tile`, `start_image`, `vector_image` etc.
- Headless backend (records tile actions, no rendering).
- Optional GUI backend (Qt or web) — separate worktree, opt-in.

Phase 16 — LiveHost Adapter

Goal: run AutoLISP scripts through clautolisp against an actual BricsCAD or AutoCAD process.

Scope (highly speculative):

- Stdio-bridge prototype against BricsCAD (Linux / macOS are feasible without ActiveX).
- COM bridge for AutoCAD on Windows.
- Integration with the existing `autolisp-spec` probe harness.

Cross-Cutting Concerns

Coverage tracking

A new tooling target `make builtins-coverage` produces a report comparing `core-builtins` entries against the function inventory in `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org`, broken down by chapter. Each phase above closes a column of that report.

Testing strategy

- **Unit tests** (FiveAM) per builtin family.
- **Snapshot tests** against MockHost: a script runs, the post-condition entity table is dumped, and the dump is compared against a stored `.snapshot` file.
- **Probe replay** against the BricsCAD V26 / macOS harness for semantics that are not directly testable headlessly (used at spec-time, not at every build).
- **File-compat scenarios** (`autolisp-file-compat`) continue to function as a declarative scenario corpus.

Compatibility profile vs dialect

The dialect descriptor (`strict`, `autocad-2026`, `bricscad-v26`) describes the **language**. The HAL backend describes the **host**. They are independent: a script under the `autocad-2026` dialect can run against MockHost; a script under `strict` can run against LiveHost. Tests must exercise the cross product.

Versioning

`clautolisp/tools/clautolisp/source/version.lisp` bumps DEVELOP on every source change. Phase boundaries optionally bump MINOR (0.0.x → 0.1.0 → ...). MAJOR is reserved for API-breaking changes (e.g., a HAL contract revision).

Open Architectural Questions

These are recorded here so they are remembered when their phase arrives:

1. **ENAME identity across sessions.** If LiveHost reconnects, are ENAMEs from a previous session re-resolvable? Probably no: every session gets a fresh ID space.
2. **Concurrent sessions.** Today's runtime is single-session. Is a pool of MockHosts useful for parallel test execution? Likely yes — `runtime-session` already isolates sessions; the open work is making MockHost itself thread-safe.
3. **Garbage collection of pickset / VLA tables.** A long-running session leaks. Provide an explicit `clear-host-handles` helper on MockHost; let LiveHost translate it to `object-released-p` sweeps.
4. **`(command ...)` interactivity in headless mode.** The `command` builtin in real AutoCAD drives the command line. In MockHost we record commands to a log; `initget` / `getstring`-like prompts feeding into `command` need a coherent contract.
5. **DCL rendering.** Headless DCL records actions. A production GUI backend will need to choose between Qt, web, or platform-native; the HAL

contract should be neutral on that question.

6. **COM type library scope.** A real AutoLISP corpus references a long tail of `vla-*` functions generated from imported type libraries. We will have to draw a line between "documented" and "user-imported" and only ship the documented surface in core.

Cross-References

- **Spec**: `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org` — chapter index drives every phase.
- **Existing PLAN**: `clautolisp/PLAN.md` — actionable tasks. This roadmap is the parent document.
- **Sibling design notes**: `clautolisp/documentation/development-plan.org` — earlier high-level architecture rationale.
- **User manual**: `clautolisp/documentation/user-manual.org` — the CLI surface, kept in sync with `tools/clautolisp/source/main.lisp`.